

MCP-Backed Chat: Angular Frontend Architecture Plan

v3

Revised against the real Angular project at `C:\Users\katlego.gagoopane\Documents\2026\Claude\frontend` |
Prepared 2026-08-13

What changed from v2: v2 assumed a fresh Angular app would be built from scratch. In fact an Angular 22, signals-based, zoneless project already exists at this path, with its own coding rules, reference screenshots of the target sign-in/chat UI, and a **separate** `angular-cli` MCP server already connected — a dev-time tooling aid for Claude Code while writing this frontend, not part of the runtime chat pipeline. The project also ships an empty Express skeleton under `server/`, but per direction this plan does **not** use it: the Chat Bridge from v2 remains the single backend the Angular app talks to, exactly as v2 originally proposed, now handling end-user auth itself as well.

1. Objective

Extend the existing Angular frontend project so that chat queries typed in the browser reach Claude (via the Chat Bridge and MCP Gateway from v2), following every convention this project already commits to in its own `CLAUDE.md` files, and reproducing the auth UX already implied by its reference screenshots — without introducing the project's Express skeleton into the request path.

2. What This Frontend Project Already Establishes

Aspect	Current state	Source
Frontend stack	Angular 22, signals-based, zoneless, standalone components (no NgModules, no explicit <code>standalone: true</code>)	<code>package.json</code> , <code>CLAUDE.md</code> , <code>.claude/CLAUDE.md</code>
Forms	Signal Forms only — no reactive or template-driven forms	<code>src/CLAUDE.md</code>
Async style	No RxJS/Observables unless explicitly required; service APIs return Promises, use <code>async/await</code> ; convert any Observable-based Angular API via <code>firstValueFrom</code>	<code>src/CLAUDE.md</code>
Templates/styles	External template/style files only, no inline; no <code>ngClass</code> / <code>ngStyle</code> (use <code>class</code> / <code>style</code> bindings)	<code>src/CLAUDE.md</code> , <code>.claude/CLAUDE.md</code>
Components/state	<code>input()</code> / <code>output()</code> / <code>computed()</code> , <code>OnPush</code> , <code>inject()</code> , lazy-loaded feature routes, accessibility must pass AXE + WCAG AA	<code>.claude/CLAUDE.md</code>
HTTP convention	All backend calls use a URL starting with <code>/api</code> ; dev proxy currently forwards <code>/api</code> → <code>http://localhost:9000</code> (repointed below)	<code>src/CLAUDE.md</code> , <code>proxy.conf.json</code>

Express skeleton	<code>server/</code> exists with only a <code>CLAUDE.md</code> , no code written — not used by this plan ; the Chat Bridge takes over the backend role instead	<code>server/CLAUDE.md</code>
Target UI	Reference screenshots already present: sign-in, sign-up, empty chat state, chat with ongoing history	<code>ui-screenshots/</code>
Angular CLI MCP server	Already connected (<code>.mcp.json</code>) — gives Claude Code live access to current Angular docs/APIs/best practices and a dev-server tool while <i>writing</i> this app. Build-time aid only, unrelated to the runtime chat/tool pipeline below.	<code>.mcp.json</code> , <code>prompts/Angular-MCP-server-instructions.md</code>

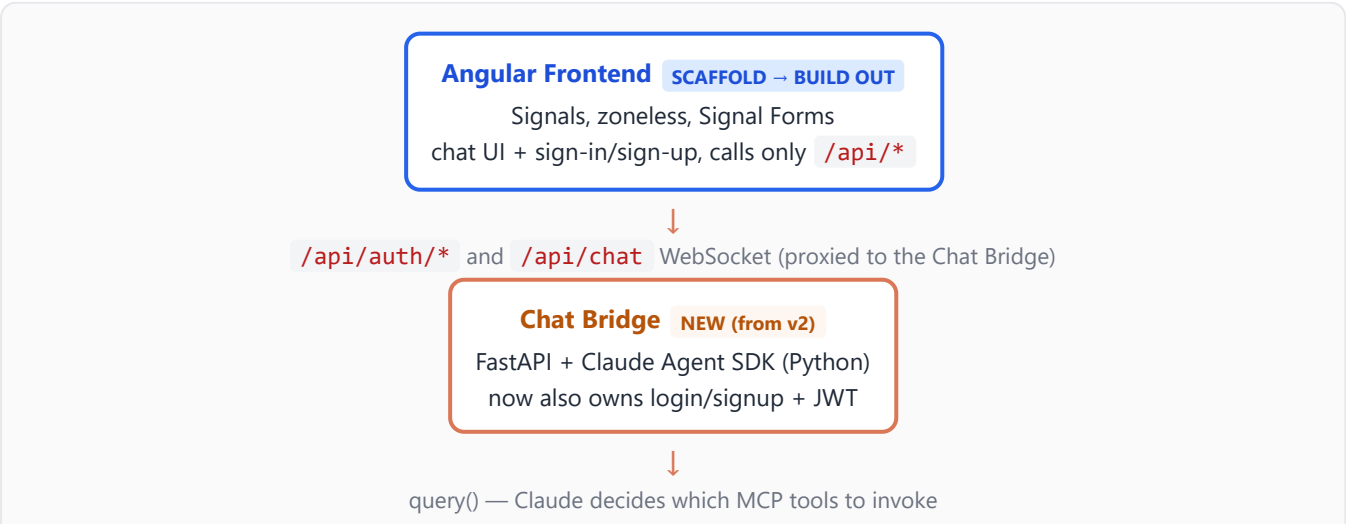
Note: `.claude/CLAUDE.md` says "prefer Reactive forms"; `src/CLAUDE.md` says "always use Signal Forms, don't use reactive or model-driven forms." `src/CLAUDE.md` is the more specific, src-scoped instruction and matches the project's own course description, so this plan follows Signal Forms — confirm intent before building the chat input if this matters.

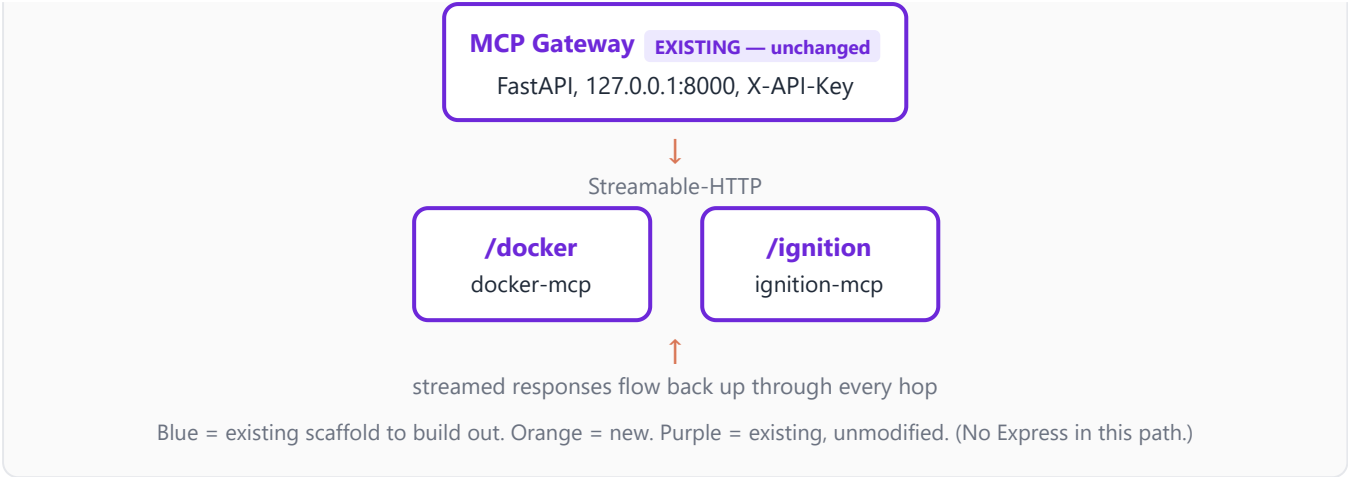
3. Recommended Architecture (v3)

This reverts to v2's shape: the browser talks directly to the Chat Bridge, and nothing else is added in between. The only change from v2 is that the Chat Bridge now also owns end-user authentication (login/signup against a mock user, JWT issuance), a responsibility that would otherwise have sat with Express.

Key decision: the Chat Bridge (FastAPI + Claude Agent SDK, Python, from v2) is the **only** backend this Angular app talks to. Repoint `proxy.conf.json`'s `/api` target from `:9000` to the Chat Bridge's port, so the frontend's own "`/api` only" rule is preserved without Express in the path. The Chat Bridge adds simple auth endpoints (e.g. `/api/auth/login`, `/api/auth/signup`) alongside the existing chat/query logic, using the same mock-user pattern (`test@angular-university.io` / `Angular123`) implied by the project's sign-in/signup screenshots. The MCP Gateway stays exactly as in v2, on an internal network the browser never reaches directly.

4. System Flow Diagram





5. Component Breakdown

Component	Status	Responsibility
Angular app (this repo)	Scaffold → build out	Sign-in/sign-up screens, chat screen with streaming display and tool-call indicators, feature-lazy-loaded, Signal Forms, no RxJS
Chat Bridge	New (Python, extends v2)	Login/signup + JWT (mock user), runs <code>claude-agent-sdk query()</code> per conversation, streams <code>SDKMessage</code> events back — the single backend this frontend talks to
MCP Gateway	Existing, unchanged	Exposes <code>/docker</code> and <code>/ignition</code> , unchanged from today
docker-mcp / ignition-mcp	Existing, unchanged	Actual tool implementations
<i>Express skeleton (<code>server/</code>)</i>	<i>Not used</i>	<i>Left in place in the repo but out of scope for this integration</i>

6. Angular-Specific Build Notes

- **Chat feature module:** a lazy-loaded `chat` route (matches "implement lazy loading for feature routes"), with a `ChatService` (`providedIn: 'root'` , `inject()`) wrapping the native `WebSocket` API in a Promise/async-friendly interface — no RxJS, so don't reach for `rxjs/webSocket` ; push incoming frames into a signal via `update()` .
- **Auth service:** an `AuthService` calling the Chat Bridge's `/api/auth/login` / `/api/auth/signup` with `fetch` / `HttpClient` wrapped as a Promise, storing the returned JWT (httpOnly cookie if the Bridge sets one, otherwise in-memory) for the subsequent chat WebSocket connection.
- **Chat input:** build with Signal Forms per `src/CLAUDE.md` , not `FormGroup` / `ReactiveFormsModule` .
- **Streaming render:** an `OnPush` , standalone `ChatMessageList` component driven by a `computed()` signal over the message array; a visibly distinct (via `class` binding, not `ngClass`) state while a Docker/Ignition tool call is in progress.
- **Accessibility:** the streaming answer region needs an `aria-live="polite"` region so screen readers announce incoming tokens/tool-status without re-reading the whole transcript; verify with AXE per

the project's existing accessibility rule.

- **External files only:** every new component gets its own `.html` / `.scss` , per `src/CLAUDE.md` .
- **Use the Angular CLI MCP server while building:** since it's already connected, use it (from within Claude Code, in this repo) to scaffold components/routes and to check current Angular 22 APIs before writing signal/forms/routing code, rather than relying on possibly-outdated training knowledge.

7. Step-by-Step Implementation Plan

1. **Stand up the Chat Bridge as designed in v2.** New Python/FastAPI service, `claude-agent-sdk` , `mcpServers` pointed at the existing Gateway's `/docker` and `/ignition` with the `X-API-Key` header held server-side.
2. **Add auth endpoints directly on the Chat Bridge.** `/api/auth/login` and `/api/auth/signup` , argon2-equivalent hashing (e.g. `passlib` / `argon2-cffi`), JWT issuance, a seeded mock user matching the existing screenshots (`test@angular-university.io` / `Angular123`), no persistent DB for now.
3. **Add JWT verification to the chat WebSocket.** The Bridge's `/api/chat` endpoint checks the same JWT issued at login before allowing a `query()` session to start.
4. **Repoint the Angular dev proxy.** Update `proxy.conf.json` so `/api` forwards to the Chat Bridge's port instead of `:9000` ; confirm the proxy also forwards WebSocket upgrades, not just plain HTTP.
5. **Build the Angular `AuthService` and sign-in/sign-up screens.** Match the existing `ui-screenshots/sign-in-screen.png` and `sign-up-screen.png` .
6. **Build the Angular `ChatService` and chat UI.** Signal Forms input, streaming message list, tool-call-in-progress indicator, matching `ui-screenshots/chat-initial-state-no-history.png` and `chat-conversation-ongoing-history-filled.png` .
7. **Wire routing.** Fill in `app.routes.ts` 's `/home` target and add the lazy-loaded chat route; add an auth guard so `/chat` requires a valid session.
8. **End-to-end test locally.** `ng serve` (port 4200) + Chat Bridge + existing MCP Gateway running together; verify login → chat → a real Docker or Ignition tool call → streamed answer back to the browser.
9. **Test edge cases.** Expired/invalid JWT on the WebSocket upgrade, Chat Bridge or Gateway unreachable, dropped WebSocket + reconnect, concurrent browser tabs/sessions for the same user.
10. **Deploy.** Angular production build served statically (e.g. via a lightweight static server or CDN) or by the Chat Bridge itself; Chat Bridge and MCP Gateway composed together (extending the existing `docker-compose.yml`); only the Chat Bridge's port reaches the browser/reverse proxy — the Gateway stays internal, as in v2.

8. Key Technical Considerations

- **Single public entry point:** the browser only ever talks to the Chat Bridge on `/api/*` , consistent with the existing convention — the MCP Gateway is never directly reachable from the browser.

- **CORS:** during local dev, `ng serve` (4200) proxies to the Bridge so the browser sees one origin; in production, decide whether Angular is served by the same origin as the Bridge or needs explicit CORS configuration on the Bridge.
- **No RxJS creep:** it's tempting to reach for `rxjs/webSocket` for a streaming connection; per this project's own rules, wrap the native `WebSocket` in a small Promise/signal-based service instead.
- **Express left dormant:** the `server/` folder and its `CLAUDE.md` remain in the repo but out of the request path; if it's reintroduced later, this plan's auth endpoints would need to move back into it.

9. Quick Reference

Item	Value
Angular dev server	<code>ng serve</code> / <code>npm start</code> — port 4200
Dev proxy	<code>proxy.conf.json</code> : <code>/api</code> → Chat Bridge port (repointed from <code>:9000</code>)
Mock user (dev)	<code>test@angular-university.io</code> / <code>Angular123</code>
Chat Bridge	New Python/FastAPI service, per v2 — the only browser-facing backend
MCP Gateway	Existing, <code>127.0.0.1:8000</code> , unchanged — internal only
Express skeleton	Present in repo (<code>server/</code>), not used by this plan
Angular CLI MCP server	Dev-time only, configured in <code>.mcp.json</code> , unrelated to the runtime chat path

10. Open Questions to Resolve Before Building

- Resolve the Signal Forms vs. Reactive Forms conflict between `.claude/CLAUDE.md` and `src/CLAUDE.md` before building the chat input/auth forms.
- Where should the JWT live client-side (httpOnly cookie set by the Bridge vs. token stored and attached manually to the WebSocket upgrade)?
- Does the mock, in-memory user model (no real DB) carry through to whatever environment this eventually runs in, or does real user storage get added later?
- Same open items as v2: per-tool confirmation UX for high-impact Docker/Ignition actions, and whether conversation history should persist beyond a single connection.

v3 — fits the Chat Bridge / MCP Gateway architecture from v2 underneath this project's existing Angular scaffold and coding conventions, without routing through the project's (unused) Express skeleton. See `mcp_frontend.pdf` (v1, generic) and `mcp_frontend_v2.pdf` (v2, real backend) for prior iterations.